Week 3 practical: a router in front of last week's extractor

BPINFOR-132, Designing, Verifying, and Shipping AI Agents. Duration: 2 teaching units, 90 minutes. Bring a laptop.

What your system gains this week

Increment three. Week 2 assumed every message was a service request to be logged. That assumption is false in any real inbox, and today's corpus is what happens when you stop pretending it is true.

Your week 2 extractor is not thrown away. It is demoted to being one specialist among five, behind a router that decides which specialist a message belongs to. That is what happens to good components in a growing system, and it is a compliment rather than a demotion.

You also add twenty four routing cases to the gold set week 2 started.

Why this session exists

The lecture argued that routing trades a classification risk for cheaper and better specialized handling, and that the trade is only defensible if you measure the classifier. Today you measure it on your own labelled set, and then you have to decide whether the trade was worth making.

The shape is the same as week 2: a control and a treatment, the same corpus, the same scorer, one variable changed. A student who finishes with a working router and no comparison against the monolith has not done the exercise.

Learning outcomes exercised

Outcome 2 (patterns and their trade-offs), outcome 6 (decomposing a problem into an architecture), outcome 12 (justifying trade-offs).

Before you arrive

Read ADP-Gulli chapters 1 to 3. Have week 2 committed, including `artifacts/goldset.json`. Warm your model: the first schema-constrained call after a cold start takes minutes, and the rest take seconds.

Timing

Time	Block	What you do
0 to 10	Definitions	TODO 1: write the five route definitions, before any code
10 to 45	Control and treatment	TODO 2, 3, 4: monolith, classifier, policy layer
45 to 70	Measure	TODO 5, 6, 7: per route counts, confusion pairs, cost, gold set
70 to 85	Stretch	TODO 8: model routing or voting, one variant per group
85 to 90	Close	Commit, push, write the numbers into `DECISIONS.md`

The corpus

Twenty four labelled messages to the same fictional commune help desk, in three languages, across five routes. Six more are held out as an example pool, exactly as in week 2.

Four of the twenty four are deliberately ambiguous. A message that reports a broken heater and complains about three weeks of silence belongs to two routes. There is no correct label there, only a documented convention, and arguing about the convention is the intended work of the first block.

Block 1, the definitions (10 minutes)

TODO 1 in `routes.py`. Ten minutes of writing before any code, and students want to skip it every year.

Define each route by what the help desk is expected to **do**, not by what the message feels like. A request can be furious and a complaint can be polite, so tone is not a route.

You may disagree with the definitions in `queries.py`. That is a legitimate choice with a consequence: your accuracy is then measured against labels written under a different convention. Decide deliberately and record it.

Block 2, control and treatment (35 minutes)

TODO 2 and 3 in `router.py`, TODO 4 in `routes.py`.

The policy layer is three lines of code and three design decisions:

- **The confidence floor.** Do not pick 0.7 because it sounds reasonable. Print the confidence values first and then choose. On one of the two course models the answer will surprise you.
- **The evidence check.** Free, inherited from week 2. Is the span actually in the message? This tests the classifier's honesty rather than its correctness, and both matter.
- **The safe default.** Where does anything the policy rejects go? Ask what each specialist *does* on the sender's behalf, and pick the one whose actions are easiest to undo.

Checkpoint 1. The six items in `checklist.md`.

Block 3, measure it (25 minutes)

TODO 5, 6, and 7.

Report per route, as counts. At four to seven gold records per route, one misroute moves that route by fifteen to twenty five points, and a percentage at that sample size is a lie told with a decimal point.

The confusion pairs matter more than the accuracy. Knowing the router is 20 of 24 tells you to try harder. Knowing that it sends requests into `info`, and only in that direction, tells you which definition is wrong.

Checkpoint 2. Be ready to say your numbers out loud, per route, as counts, with the confusion pairs named.

Block 4, stretch (15 minutes)

TODO 8. One variant per group.

- **Model routing.** Same prompt, same policy, only the model changes. Predict which wins before you run it.
- **Voting.** Three samples at temperature 0.7, majority wins. The accuracy is not the interesting output.

Working with the recording

```
python starter/01_compare.py --replay
```

Every call the session makes is recorded, including both models and the voting samples. Develop against it, then run live.

The recording contains real failures, unedited: two routes that collapse into `info`, evidence spans that get paraphrased on one of the two models, and a confidence signal that turns out to be useless on the other. If your scorer reports everything correct, your scorer does nothing.

What goes into DECISIONS.md today

Six entries. Template in `starter/DECISIONS_week03_section.md`.

1. The five route definitions, and your convention for the ambiguous cases.
2. The policy: your threshold, the evidence check, the safe default, and why each. Include what you saw in the confidence distribution before choosing.
3. Route accuracy per route as counts, plus the confusion pairs and their direction, with the model name and the run date.
4. The routing overhead: extra call, extra tokens, and its share of the total.
5. What each specialist can be forbidden to do that the monolith cannot, and whether you would ship the router on that basis.
6. Your stretch variant, its result, or an explicit deferral with a reason.

Homework

- Run the whole comparison live if you only used the recording.
- Wire your week 2 extractor in behind the `request` route, so that route produces a validated record rather than prose. This is the first place the two weeks become one system.
- `python -m project.verify` passing, with the gold set now holding thirty four cases.

If you finish early

- Rewrite the one definition your confusion pairs point at, rerun, and report whether the pair moved. Changing a definition and measuring the effect is a different skill from changing a prompt, and it is the more valuable one.
- Time the three voting samples sequentially and then in a thread pool. Explain why the parallel version is not three times faster against a local endpoint, and what that says about when parallelization is worth it.
- Add a query of your own that you believe is genuinely unroutable, and see what your policy layer does with it.

Reference solution

In `solution/`, published after the session. The written answers to TODO 7 and TODO 8 are at the bottom of `01_compare.py` and `02_stretch.py`.